

PONTIFICIA UNIVERSIDAD CATOLICA DEL ECUADOR

Facultad de Ingenieria

Ingenieria en Sistemas y Computacion

PROYECTO P1.2

Comparacion del Rendimiento entre CPU y GPU en el Procesamiento Paralelo de Cadenas de ADN

Asignatura: Computacion Paralela, Distribuida y en la Nube

Docente: Ing. JW Condor

Autor: Alex Vivanco

1. Resumen

Este informe presenta el desarrollo de una aplicación en Python que procesa el archivo FASTA del genoma humano GRCh38.p14, de 3,11 GB, comparando el rendimiento de dos plataformas de cómputo paralelo: la CPU multinúcleo mediante multiprocessing y la GPU mediante kernels CUDA escritos con Numba. Además del conteo de los nucleótidos A, C, G y T, la aplicación identifica los caracteres inválidos presentes en la secuencia y los distingue de los códigos IUPAC de ambigüedad, que son notación válida del formato.

Sobre el archivo completo, la ejecución secuencial en CPU tomó 51.463 segundos, la ejecución paralela con doce procesos tomó 17.469 segundos y la ejecución en GPU tomó 3.443 segundos, lo que representa un speedup de 14.95 veces frente a la versión secuencial y de 5.07 veces frente a la mejor configuración de CPU. Las tres ejecuciones produjeron conteos idénticos, verificados sobre el histograma completo de 256 valores de byte y no únicamente sobre las cuatro bases.

2. Planteamiento del problema

El ADN se representa computacionalmente como una cadena de caracteres construida sobre un alfabeto de cuatro símbolos que corresponden a las bases nitrogenadas adenina, citosina, guanina y timina. El conteo de frecuencias sobre esa cadena es una operación algorítmicamente trivial, pero deja de serlo cuando el volumen de datos alcanza varios gigabytes, porque el costo deja de estar en el cálculo y pasa a estar en el acceso a los datos.

El proyecto P1.1 resolvió este problema repartiendo el trabajo entre los núcleos de la CPU. El presente proyecto plantea una pregunta distinta: si una GPU, diseñada para ejecutar miles de hilos simples en paralelo, resulta más adecuada que la CPU para este tipo de carga, y bajo qué condiciones. La respuesta no es evidente de antemano, porque la GPU introduce un costo que la CPU no tiene: los datos deben viajar por el bus PCIe desde la memoria principal hasta la memoria de la tarjeta antes de poder ser procesados.

El archivo utilizado sigue el formato FASTA, en el que las líneas que comienzan con el símbolo mayor que corresponden a cabeceras descriptivas y deben excluirse del conteo. Las líneas de secuencia contienen además la letra N para bases desconocidas y emplean minúsculas como marcado de regiones repetitivas, que deben contarse igual que las mayúsculas.

3. Objetivos

- Desarrollar una aplicación que procese un archivo FASTA de varios gigabytes empleando paralelismo sobre GPU mediante CUDA.
- Implementar una versión equivalente sobre CPU multinúcleo que sirva de término de comparación válido.
- Identificar los caracteres inválidos presentes en la secuencia y distinguirlos de los códigos válidos del formato.

- Medir y comparar el desempeño de ambas plataformas en terminos de tiempo de ejecucion, velocidad de procesamiento y uso de recursos.
- Determinar el tamano de lote optimo para el balance de carga de la GPU.
- Analizar la escalabilidad de ambas plataformas frente a volúmenes crecientes de datos e identificar los cuellos de botella.

4. Ambiente de pruebas

Todas las mediciones se realizaron sobre el mismo equipo y bajo las mismas condiciones. Las especificaciones son las siguientes:

- Procesador: Intel Core i5-13420H, 13.a generacion, arquitectura hibrida.
- Nucleos fisicos: 8. Procesadores logicos: 12.
- Tarjeta grafica: NVIDIA GeForce RTX 4050 Laptop GPU, arquitectura Ada Lovelace, compute capability 8.9, 20 multiprocesadores de flujo (SM), 6140 MB de memoria VRAM, tamano de warp de 32 hilos.
- Memoria RAM: 32 GB. Almacenamiento: unidad de estado solido NVMe.
- Sistema operativo: Windows 11.
- Lenguaje: Python 3.13, con las librerias numba 0.67 y numba-cuda 0.30 para la GPU, multiprocessing para la CPU, y psutil, pynvml, matplotlib y tkinter para el monitoreo y la interfaz.
- Archivo de entrada: GCF_000001405.40_GRCh38.p14_genomic.fna, de 3,11 GB.

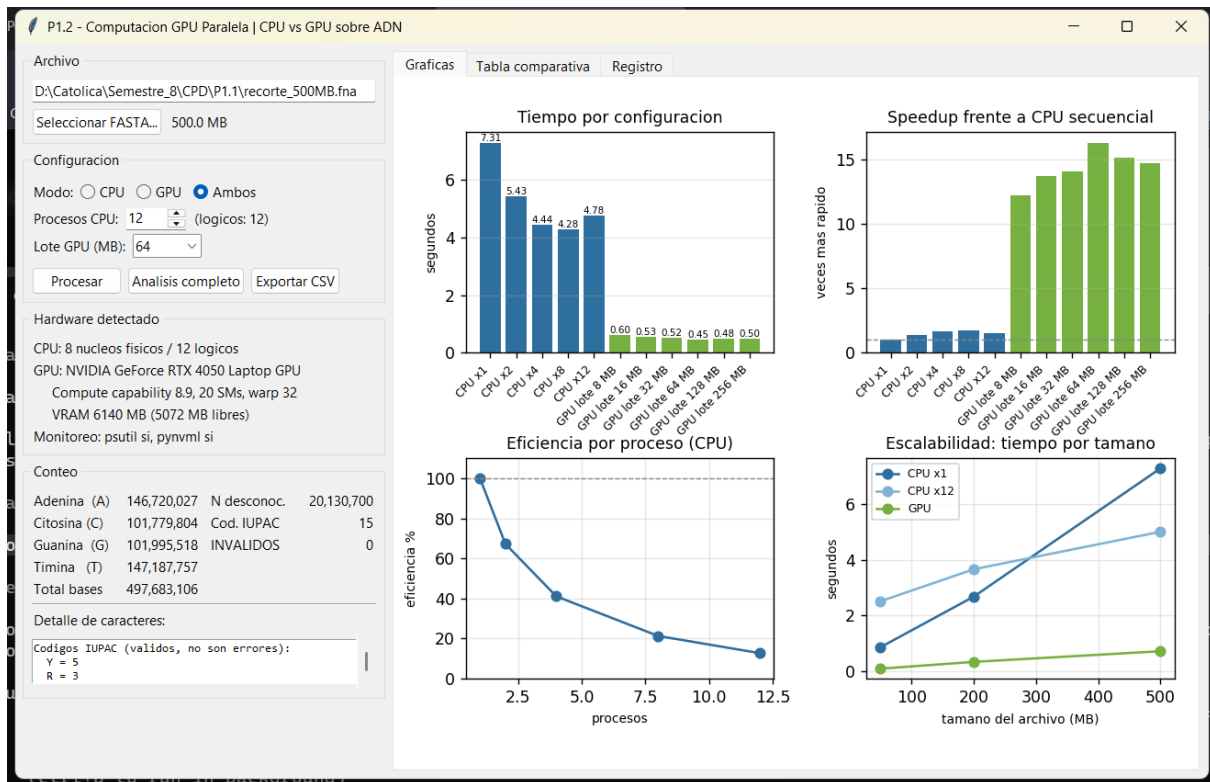


Figura 1. Aplicacion desarrollada, con el hardware detectado en el panel izquierdo y los resultados del analisis completo.

5. Diseno de la solucion

5.1 Un mismo algoritmo sobre dos plataformas

La decision de diseno mas importante del proyecto fue que ambos motores ejecutaran exactamente el mismo algoritmo: el calculo de un histograma completo de 256 posiciones, una por cada valor posible de byte, del que se derivan despues todos los conteos. Si la CPU emplease un atajo distinto al de la GPU, los tiempos medidos no serian comparables y la comparativa perderia todo su valor. De este modo la unica variable entre una medicion y otra es el hardware que ejecuta, no el algoritmo.

5.2 Descarte de cabeceras dentro de la GPU

Las lineas de cabecera deben excluirse del conteo. La solucion sencilla seria limpiarlas en el procesador antes de enviar el lote a la tarjeta, pero eso significaria que la CPU realiza parte del trabajo de la GPU y falsearia la comparacion. Por ello el descarte se resuelve integramente dentro de la tarjeta, mediante tres kernels encadenados:

- El primer kernel asigna un hilo a cada byte del lote y marca las posiciones donde comienza una cabecera, es decir, aquellos simbolos mayor que precedidos de un salto de linea. Las posiciones se acumulan mediante una operacion atomica sobre un contador compartido.
- El segundo kernel asigna un bloque a cada cabecera encontrada. El primer hilo del bloque localiza el final de la linea y el resto de hilos la sobreescibe en paralelo con saltos de linea, que el histograma ya ignora. Las cabeceras son pocas y cortas, de modo que el costo es despreciable.
- El tercer kernel calcula el histograma. Cada bloque acumula primero en un vector de 256 posiciones alojado en memoria compartida, que reside dentro del multiprocesador y es mucho mas rapida que la memoria global, y solo al terminar vuelca su resultado parcial al histograma global. Sin esta acumulacion intermedia, millones de hilos competirian atomicamente por las mismas cuatro posiciones y el kernel se serializaria.

El segundo kernel se lanza con una malla de tamano fijo y cada bloque consulta en memoria de dispositivo cuantas cabeceras hay que procesar. Esto evita tener que copiar ese numero al procesador entre un kernel y el siguiente, operacion que obligaria a sincronizar y romperia el solapamiento entre transferencia y computo.

5.3 Procesamiento por lotes y solapamiento

El archivo ocupa 3,11 GB y la tarjeta dispone de 6140 MB de memoria, de los cuales el sistema operativo ya reserva una parte. Cargar el archivo completo no es viable. La aplicacion lo procesa por lotes, empleando dos buferes de memoria de pagina bloqueada, requisito para que las transferencias sean asincronas, y dos flujos CUDA que se alternan. De este modo la transferencia de un lote se solapa con el computo del lote anterior. El tamano de lote es

configurable, ya que constituye el parametro de balance de carga evaluado en la seccion de resultados.

5.4 Separacion de la compilacion del tiempo medido

Numba compila cada kernel la primera vez que se invoca, operacion que en este equipo tarda entre 1,9 y 2,8 segundos. Si ese tiempo se incluyera en la medicion, se le estaria atribuyendo a la GPU un costo que no pertenece al algoritmo, y en archivos pequenos la compilacion pesaria mas que el procesamiento. La aplicacion compila los kernels de forma explicita antes de iniciar cualquier cronometro.

6. Implementacion

La solucion se estructuro en modulos independientes. El motor de CPU y el motor de GPU exponen la misma interfaz y devuelven el mismo diccionario de resultados. Un modulo de monitoreo muestrea el uso de recursos en un hilo de fondo; un generador produce archivos de prueba con errores inyectados; un banco de pruebas ejecuta los barridos comparativos; y la interfaz grafica reune todo lo anterior.

El procesamiento se ejecuta siempre en un hilo separado del hilo de la interfaz. Dado que Tkinter no admite el acceso a sus componentes desde varios hilos, el hilo trabajador nunca modifica un componente de forma directa: deposita mensajes en una cola que la ventana vacia periodicamente. Este esquema es el que permite que la aplicacion siga respondiendo mientras procesa un archivo de tres gigabytes.

Un detalle relevante del comportamiento en Windows condiciona la arquitectura de archivos. En este sistema multiprocessing emplea el metodo spawn, de modo que cada proceso hijo vuelve a importar el modulo principal. Durante el desarrollo se midio que la simple presencia de la importacion del motor de GPU en el modulo principal elevaba el tiempo de una ejecucion paralela sobre un archivo de 2 MB de 1,34 a 5,51 segundos, porque los doce procesos hijos cargaban la infraestructura de CUDA sin llegar a utilizarla. Ese costo se le habria atribuido erroneamente a la CPU. Por ello el archivo de arranque de la aplicacion contiene unicamente el lanzador, y tanto la ventana como los motores se importan de forma diferida.

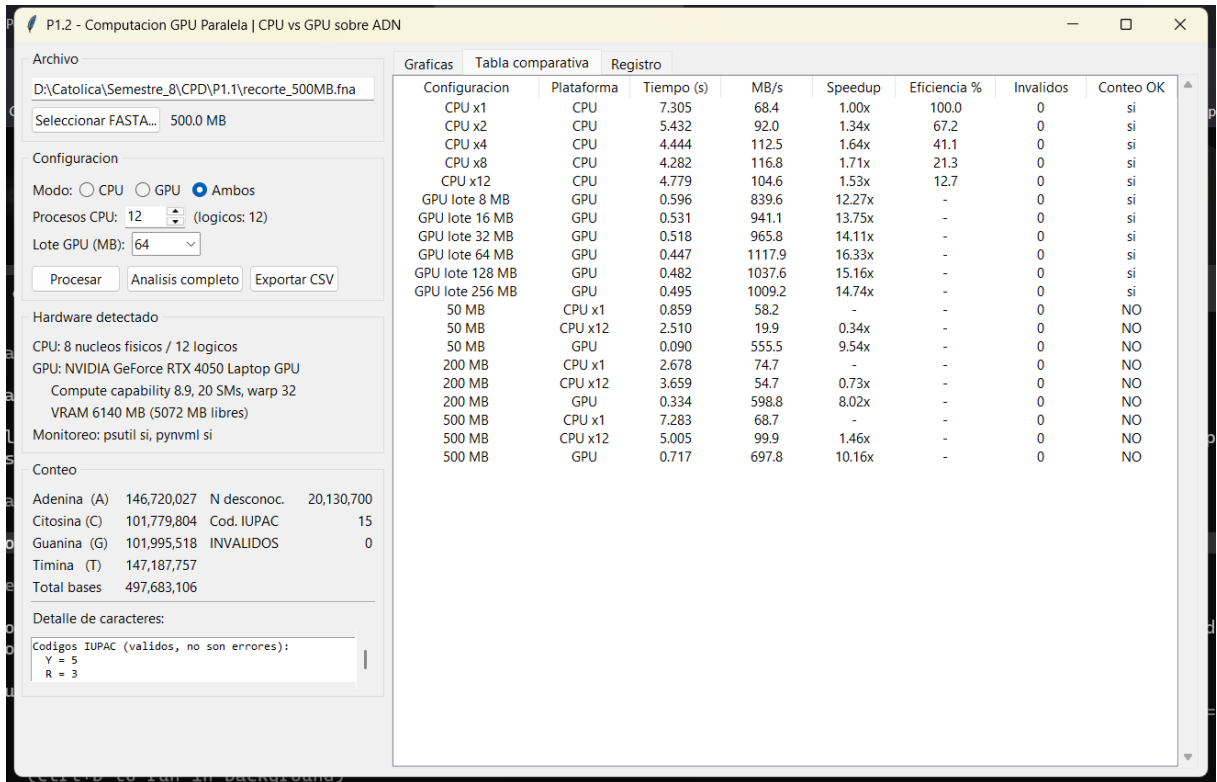


Figura 2. Tabla comparativa de la aplicacion con todas las configuraciones medidas y la verificacion de conteos.

7. Resultados

7.1 Conteo de nucleotidos

El procesamiento del archivo completo arrojo los siguientes resultados, identicos en las tres modalidades de ejecucion:

Base	Ocurrencias	Porcentaje	Par complementario
Adenina (A)	923.117.203	29.43 %	Timina
Citosina (C)	642.552.917	20.48 %	Guanina
Guanina (G)	645.231.996	20.57 %	Citosina
Timina (T)	925.917.038	29.52 %	Adenina
Total de bases	3.136.819.154	100,00 %	-
N (desconocidas)	161.611.379	-	No aplica

Los valores obtenidos permiten una validacion adicional del resultado. Dado que el ADN es una molecula de doble hebra en la que la adenina se empareja siempre con la timina y la citosina con la guanina, en un genoma completo ambas cantidades deben ser practicamente iguales. La diferencia observada entre adenina y timina es de apenas el 0.30 por ciento, y entre citosina y guanina del 0.42 por ciento, lo que confirma la correctitud del conteo por una via independiente del propio programa.

7.2 Identificacion de caracteres invalidos

El analisis del archivo completo revelo 103 caracteres que no son ni las cuatro bases ni la letra N. Un examen de su naturaleza mostro que no se trata de errores sino de codigos IUPAC de ambigüedad, notacion estandar y valida del formato FASTA para posiciones en las que la secuenciacion no resolvió una base concreta pero si acoto las posibilidades:

Codigo	Ocurrencias	Significado
Y	36	C o T (pirimidina)
R	29	A o G (purina)
W	15	A o T (enlace debil)
K	8	G o T (ceto)
M	8	A o C (amino)
S	5	G o C (enlace fuerte)
B	2	C, G o T (no A)

Clasificar estos codigos como caracteres invalidos habria constituido un error conceptual, ya que no son basura sino informacion parcial. Por ello la aplicacion los contabiliza en una categoria propia, separada tanto de las bases como de los errores.

Ahora bien, el archivo real no contiene ningun caracter invalido en sentido estricto, por lo que no permite demostrar que el detector funciona. Para obtener una prueba verificable se desarrollo un generador que produce copias del genoma con una cantidad y un tipo de error conocidos de antemano, sustituyendo caracteres de las lineas de secuencia sin alterar el tamaño ni la estructura del archivo. Se contemplan cuatro familias de error: letras ajenas al alfabeto del ADN, digitos, simbolos y espacios en blanco, siendo estos ultimos los mas dificiles de advertir porque no son visibles al abrir el archivo. Sobre un archivo de prueba con 1094 errores inyectados de 24 tipos distintos, las tres modalidades de ejecucion reprodujeron exactamente la cantidad y el desglose esperados.

7.3 Desempeno sobre el archivo completo

La tabla siguiente resume las mediciones sobre el genoma completo de 3,11 GB. El speedup se calcula respecto de la ejecucion secuencial.

Configuracion	Tiempo (s)	MB/s	Speedup	Conteo identico
CPU secuencial	51.463	61.9	1,00x	Si
CPU paralelo x12	17.469	182.3	2.95x	Si
GPU (lote 128 MB)	3.443	925.2	14.95x	Si

7.4 Barrido de configuraciones

Para caracterizar el comportamiento de ambas plataformas se realizo un barrido detallado sobre un recorte de 500 MB del mismo archivo, que permite repetir muchas configuraciones en un tiempo razonable.

Configuracion	Tiempo (s)	MB/s	Speedup	Eficiencia
CPU x1 (secuencial)	7.305	68.4	1.00x	100.0 %
CPU x2	5.432	92.0	1.34x	67.2 %

CPU x4	4.444	112.5	1.64x	41.1 %
CPU x8	4.282	116.8	1.71x	21.3 %
CPU x12	4.779	104.6	1.53x	12.7 %

Configuracion	Tiempo (s)	MB/s	Speedup
GPU lote 8 MB	0.596	839.6	12.27x
GPU lote 16 MB	0.531	941.1	13.75x
GPU lote 32 MB	0.518	965.8	14.11x
GPU lote 64 MB	0.447	1117.9	16.33x
GPU lote 128 MB	0.482	1037.6	15.16x
GPU lote 256 MB	0.495	1009.2	14.74x

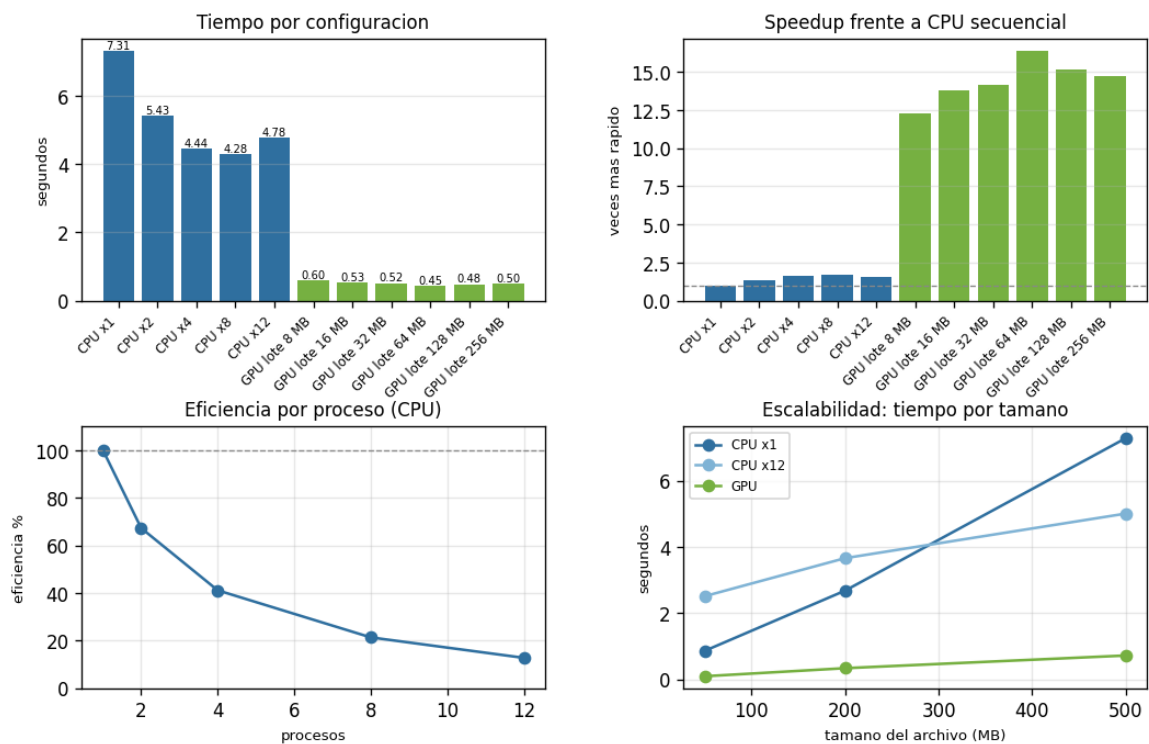


Figura 3. Tiempo por configuracion, speedup, eficiencia de la CPU y escalabilidad de ambas plataformas.

7.5 Escalabilidad

El ultimo barrido midio el comportamiento de ambas plataformas frente a volúmenes crecientes de datos, con el proposito de determinar a partir de que tamaño compensa el costo fijo de la GPU.

Tamano	CPU x1 (s)	CPU x12 (s)	GPU (s)	GPU frente a CPU x1
50 MB	0.859	2.510	0.090	9.54x
200 MB	2.678	3.659	0.334	8.02x

500 MB	7.283	5.005	0.717	10.16x
--------	-------	-------	-------	--------

8. Analisis critico

8.1 Origen de la ventaja de la GPU

La GPU resulto 14.95 veces mas rapida que la ejecucion secuencial y 5.07 veces mas rapida que la mejor configuracion de CPU sobre el archivo completo. La razon de esta diferencia esta en la naturaleza del problema. El conteo de frecuencias es una operacion sin dependencias entre elementos: el resultado de un caracter no condiciona el de ningun otro. Esta característica se ajusta con precision al modelo de ejecucion de una GPU, que dispone de veinte multiprocesadores capaces de mantener miles de hilos simultaneos ejecutando la misma instruccion sobre datos distintos. La CPU, en cambio, dispone de doce procesadores logicos que ademas deben atender al sistema operativo.

8.2 El verdadero cuello de botella

La velocidad maxima alcanzada por la GPU fue de 1117,9 MB/s sobre el recorte de 500 MB y de 925,2 MB/s sobre el archivo completo. Estas cifras no reflejan la capacidad de calculo de la tarjeta sino la velocidad de lectura del disco de estado solido. El kernel de histograma procesa los datos mucho mas rapido de lo que el subsistema de almacenamiento consigue suministrarlos, de modo que durante la mayor parte de la ejecucion la GPU permanece a la espera.

Esta observacion tiene una consecuencia importante para la interpretacion de los resultados: el speedup medido no expresa la superioridad computacional de la GPU sobre la CPU, sino unicamente que la GPU alcanza antes el limite impuesto por el almacenamiento. Si el archivo residiera integramente en memoria, la diferencia entre ambas plataformas seria considerablemente mayor.

8.3 Tamano de lote y balance de carga

El barrido del tamano de lote mostro un optimo claro en 64 MB, con 1117,9 MB/s, frente a los 839,6 MB/s obtenidos con lotes de 8 MB y los 1009,2 MB/s con lotes de 256 MB. El comportamiento responde a dos efectos opuestos. Con lotes pequenos se desaprovecha el ancho de banda del bus PCIe y se multiplica el numero de lanzamientos de kernel, cada uno con su propia latencia de inicio. Con lotes grandes, en cambio, el procesador debe llenar por completo el bufer antes de que la tarjeta pueda comenzar a trabajar, lo que retrasa el inicio del computo y reduce el solapamiento efectivo entre transferencia y calculo. El valor de 64 MB equilibra ambos efectos en este equipo.

8.4 Caída de la eficiencia en la CPU

La eficiencia de la CPU descendio de forma sostenida desde el 100 por ciento con un proceso hasta el 12,7 por ciento con doce. El motivo es que el problema esta limitado por la entrada y

salida de datos, no por el calculo: anadir procesos incorpora unidades de ejecucion, pero no anade ancho de banda de disco. A partir de ocho procesos el tiempo dejo incluso de mejorar y con doce empeoro, pasando de 4,282 a 4,779 segundos, porque los cuatro procesos adicionales compiten por los mismos recursos fisicos.

A ello se suma la naturaleza hibrida del procesador empleado. Los ocho nucleos fisicos no son equivalentes entre si: parte de ellos son nucleos de rendimiento y el resto nucleos de eficiencia, de menor capacidad. Al pasar de ocho a doce procesos no se incorporan nucleos adicionales sino hilos logicos que comparten recursos con los ya activos.

8.5 Cuando la paralelizacion resulta contraproducente

El barrido de escalabilidad dejo al descubierto un resultado que conviene destacar. Sobre el recorte de 50 MB, la ejecucion paralela con doce procesos tardo 2,510 segundos frente a los 0,859 segundos de la ejecucion secuencial, es decir, resulto casi tres veces mas lenta. La causa es el costo de creacion de los procesos: en Windows cada proceso hijo arranca un interprete nuevo y vuelve a importar los modulos necesarios, operacion que sobre un archivo pequeno consume mas tiempo que el propio procesamiento. Solo a partir de los 500 MB la version paralela recupera la ventaja. La GPU, por el contrario, se mantuvo por delante en los tres tamanos evaluados, porque su costo fijo se limita a la transferencia y no incluye la creacion de procesos del sistema operativo.

8.6 Consideracion metodologica

El equipo dispone de 32 GB de memoria RAM y el archivo procesado ocupa 3,11 GB, por lo que el sistema operativo puede mantener una fraccion considerable del archivo en su cache de disco entre una ejecucion y la siguiente. Las mediciones sucesivas de un mismo barrido se benefician por tanto de lecturas mas rapidas que la primera. Dado que este efecto favorece por igual a ambas plataformas, no invalida la comparacion, pero si implica que las cifras absolutas de velocidad deben interpretarse como un limite superior optimista.

9. Aplicaciones reales

El escenario planteado en este proyecto constituye una version simplificada de problemas que se presentan de forma cotidiana en bioinformatica. El alineamiento de secuencias contra un genoma de referencia, la deteccion de variantes geneticas, el calculo del contenido de guanina y citosina por regiones o el control de calidad de los datos producidos por un secuenciador operan sobre volúmenes comparables o superiores.

Estas operaciones comparten la caracteristica observada aqui: son algorítmicamente simples pero se aplican sobre cantidades de datos muy grandes, lo que las convierte en candidatas naturales para la paralelizacion en GPU. De hecho, herramientas de uso extendido en el sector, como los alineadores acelerados por hardware, se fundamentan precisamente en este principio. El patron trasciende ademas el ambito biologico: el mismo esquema de histograma paralelo

sobre memoria compartida es el que sustenta el analisis de registros de servidores, el procesamiento de imagenes y la construccion de indices sobre grandes colecciones de texto.

10. Conclusiones

- La ejecucion en GPU redujo el tiempo de procesamiento del genoma completo de 51.463 a 3.443 segundos frente a la version secuencial, lo que representa un speedup de 14.95 veces, y de 17.469 a 3.443 segundos frente a la mejor configuracion de CPU, con un speedup de 5.07 veces.
- La correctitud se mantuvo en todas las configuraciones evaluadas. La verificacion se realizo sobre el histograma completo de 256 valores de byte, criterio mas exigente que la simple comparacion de las cuatro bases, y todas las ejecuciones coincidieron.
- La proporcion obtenida entre bases complementarias, con diferencias inferiores al 0,31 por ciento entre adenina y timina y al 0,42 por ciento entre citosina y guanina, valida el resultado por una via biologica independiente del programa.
- El cuello de botella del sistema no reside en el calculo sino en el acceso a los datos. La GPU alcanza el limite del almacenamiento con holgura, mientras que la CPU necesita ocho procesos para aproximarse a el.
- El tamano de lote optimo para esta tarjeta se situo en 64 MB, valor que equilibra el aprovechamiento del bus PCIe con el solapamiento entre transferencia y computo.
- La paralelizacion no resulta ventajosa de forma incondicional. Sobre archivos pequenos el costo de creacion de procesos supera al beneficio, hasta el punto de que la version paralela puede ser tres veces mas lenta que la secuencial.
- Los 103 caracteres no convencionales hallados en el genoma resultaron ser codigos IUPAC de ambigüedad y no errores, por lo que se clasificaron en una categoria propia. La capacidad de deteccion de errores reales se valido mediante archivos generados con errores conocidos.

11. Recomendaciones

- Evaluar la lectura asincrona del archivo mediante varios hilos de entrada y salida, con el fin de determinar si es posible superar el limite de velocidad impuesto por el acceso secuencial al disco.
- Repetir cada configuracion varias veces y reportar el promedio, de modo que se reduzca el efecto de la cache del sistema operativo y de la variabilidad propia de la planificacion.
- Extender la comparacion a la tarjeta grafica integrada del procesador mediante OpenCL, con el proposito de contrastar el rendimiento de una unidad discreta frente a una integrada.
- Explorar el uso de memoria de textura o de instrucciones vectorizadas de carga en el kernel de histograma, para el caso en que los datos estuvieran ya residentes en memoria y el disco dejara de ser el factor limitante.

- Considerar la implementacion del motor de CPU en un lenguaje compilado, con el objeto de establecer el limite superior real de la plataforma y obtener una comparacion mas estricta frente a la GPU.

12. Referencias

- NVIDIA Corporation. CUDA C++ Programming Guide. Disponible en: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- Anaconda Inc. Numba Documentation: CUDA Python. Disponible en: <https://numba.readthedocs.io/en/stable/cuda/index.html>
- Python Software Foundation. Documentacion oficial del modulo multiprocessing. Disponible en: <https://docs.python.org/3/library/multiprocessing.html>
- National Center for Biotechnology Information. Genome Reference Consortium Human Build 38 (GRCh38). Disponible en: https://www.ncbi.nlm.nih.gov/assembly/GCF_000001405.40/
- Cornish-Bowden, A. Nomenclature for incompletely specified bases in nucleic acid sequences. Nucleic Acids Research, vol. 13, num. 9, 1985.
- NVIDIA Corporation. Especificaciones de la arquitectura Ada Lovelace y de la serie GeForce RTX 40. Disponible en: <https://www.nvidia.com/>